

CyberShell Copilot

SmartTerminal GitHub Release Guide

Offline cybersecurity-aware terminal assistant for Kali, Parrot, and other Linux users

Repository	https://github.com/waltr0/smartTerminal
Package	cybershell-copilot
CLI commands	cybershell and cshell
Primary users	SOC analysts, Linux administrators, cybersecurity learners, lab operators
Repository scope	Application source, tests, docs, installer, CI, PDF guide. Marketing videos are kept outside GitHub.

This document explains what the project is, why it exists, how the components work, and exactly which commands a Kali or Parrot user should run to install, test, use, and uninstall the application.

1. Core Idea

CyberShell Copilot is a local terminal assistant for cybersecurity work. Its job is to help a user convert intent into safer Linux commands while detecting risky, destructive, or suspicious command patterns before execution. It is designed for defensive workflows such as SSH brute-force triage, process review, account auditing, Docker review, Kubernetes review, and system log investigation.

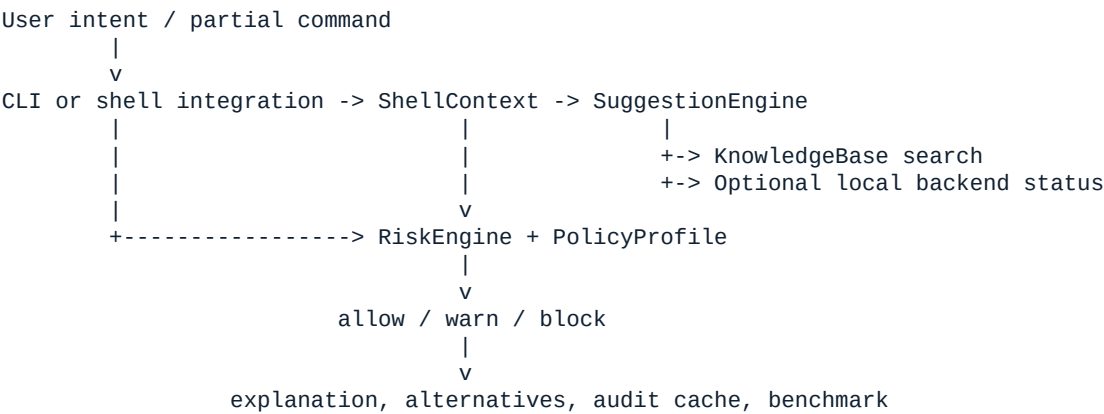
The project is intentionally local-first. The baseline engine does not require a cloud model to decide whether a command is safe, risky, or blocked. This makes it suitable for Kali, Parrot, lab machines, classroom systems, and incident-response environments where command history and local context should not be sent outside the host.

The high-level concept is simple: a user types intent or a partial command, CyberShell checks local context, retrieves a safe command candidate from the packaged knowledge base, applies policy-aware guardrails, explains the result, and optionally records accepted suggestions for auditability.

2. Why This Project Matters

Need	How CyberShell addresses it
Reduce dangerous copy-paste behavior	Blocks patterns such as piping remote downloads directly into a shell and destructive recursive deletion.
Support blue-team speed	Provides ready defensive commands for logs, processes, containers, accounts, services, and Kubernetes review.
Teach command safety	Explains why a command is safe, warned, dangerous, or blocked, including safer alternatives.
Work offline	Core rules, policies, playbooks, benchmark cases, and command records ship inside the package.
Be research-ready	Includes CyberShell-Bench, evaluation metrics, policy modes, and documented guardrail logic.

3. Architecture Overview



The command line interface is the public entry point. It builds a ShellContext from command-line arguments such as current directory, shell, history file, previous exit status, and policy mode. The SuggestionEngine searches the packaged knowledge base and sends the candidate command to the RiskEngine. The RiskEngine evaluates guardrail rules and policy thresholds, producing a decision: allow, warn, or block.

4. Main Runtime Components

Component	Responsibility	Why it exists
CLI	Parses user commands and dispatches suggest, risk, explain, audit, benchmark, and playbook actions.	Gives Linux users one stable terminal interface.

Component	Responsibility	Why it exists
Suggestion engine	Turns intent and shell context into a safe command candidate.	Makes the app useful as a copilot, not only a scanner.
Knowledge base	Stores curated command records with descriptions, tags, domains, risk levels, and ATT&CK-style mappings.	Keeps the baseline app useful offline.
Risk engine	Matches commands against guardrail rules and calculates risk score, level, decision, findings, and alternatives.	Prevents unsafe command suggestions from being shown or accepted silently.
Policy modes	Adjust thresholds for strict, soc, learner, admin, and lab use cases.	Lets one tool fit production, training, administration, and authorized lab work.
Audit and cache	Records accepted suggestions and summarizes local usage.	Creates accountability and supports later review.
Benchmark	Runs CyberShell-Bench and reports accuracy, precision, recall, and latency.	Supports regression testing and research publication.

5. File-by-File Explanation

File or folder	What it does	How it works
README.md	Main GitHub landing page.	Explains features, install flow, CLI examples, safety model, and repository structure.
pyproject.toml	Python packaging metadata.	Defines package name, version, scripts, optional dependencies, package data, and GitHub URLs.
install.sh	Linux/Kali installer.	Creates a private venv, installs the package, writes cybershell/cshell launchers, copies shell integration scripts, and runs doctor.
uninstall.sh	Clean uninstaller.	Removes launchers, shell integration blocks, app home, and optionally user data.
Makefile	Convenience commands.	Wraps install, uninstall, test, and shell-specific install actions.
.gitattributes	Git line-ending and binary rules.	Forces LF endings for shell/Python/config files and treats PDF files as binary.
.gitignore	Keeps generated files out of Git.	Ignores venvs, caches, build artifacts, coverage, and OS files.
LICENSE	MIT license.	Defines open-source use rights and warranty limits.
SECURITY.md	Security reporting policy.	Explains what to report, supported versions, and safe disclosure expectations.
CONTRIBUTING.md	Contributor guide.	Shows clone, test, benchmark, and pull-request practices.
CODE_OF_CONDUCT.md	Community behavior rules.	Sets respectful contribution expectations.
src/cybershell/cli.py	Main CLI implementation.	Builds argparse subcommands and calls engine, risk, KB, benchmark, audit, and playbook functions.
src/cybershell/engine.py	Suggestion engine.	Uses context and knowledge-base retrieval, then suppresses unsafe blocked candidates.
src/cybershell/risk.py	Risk assessment engine.	Loads guardrail rules, matches command text and context, calculates score, level, decision, findings, and alternatives.
src/cybershell/policy.py	Policy mode loader.	Reads thresholds and mode descriptions from packaged policy data.
src/cybershell/kb.py	Knowledge-base search.	Tokenizes queries and ranks command records by tag, description, domain, and command matches.
src/cybershell/data_loader.py	Packaged data access.	Loads JSON/JSONL files through importlib.resources so installed packages can find data reliably.

File or folder	What it does	How it works
src/cybershell/models.py	Data models.	Defines ShellContext, command records, risk findings, assessments, and suggestion results.
src/cybershell/alternatives.py	Safer command suggestions.	Maps risky categories to read-only or lower-risk replacement commands.
src/cybershell/audit.py	Audit helpers.	Redacts sensitive environment values and summarizes JSONL audit records.
src/cybershell/cache.py	Accepted suggestion cache.	Stores accepted suggestions by prefix so repeated workflows can be reused locally.
src/cybershell/benchmark.py	Benchmark evaluator.	Runs CyberShell-Bench cases through the risk engine and calculates accuracy, precision, recall, and latency.
src/cybershell/backends.py	Optional backend detector.	Reports whether FAISS, sentence-transformers, numpy, and llama-cpp-python are available.
src/cybershell/text.py	Text utilities.	Provides normalization/token helpers used by search and matching.
src/cybershell/__main__.py	Module execution entry.	Allows python -m cybershell to behave like the cybershell command.
src/cybershell/data/command_kb.json	Curated command records.	Contains safe Linux, Docker, Kubernetes, logging, process, and account-audit commands.
src/cybershell/data/guardrail_rules.json	Risk rules.	Contains pattern rules, scores, categories, and ATT&CK-style mappings.
src/cybershell/data/policies.json	Policy mode thresholds.	Defines strict, soc, learner, admin, and lab thresholds.
src/cybershell/data/playbooks.json	Defensive workflows.	Stores ordered read-only playbook steps for common investigations.
benchmarks/cybershell_bench.jsonl	Repository benchmark copy.	Keeps benchmark data easy to inspect and version outside the package.
scripts/cybershell.bash	Bash integration.	Adds key bindings for suggestion insertion and risk preview.
scripts/cybershell.zsh	Zsh integration.	Adds ZLE widgets and key bindings for suggestion and risk preview.
tests/*.py	Automated test suite.	Covers CLI behavior, engine suggestions, KB retrieval, and risk decisions.
docs/*.md and docs/*.pdf	Technical documentation.	Covers install, Kali setup, shell integration, architecture, security model, evaluation, benchmark schema, research artifact, and this guide.
.github/workflows/ci.yml	GitHub Actions CI.	Runs tests and benchmark checks on pushes and pull requests.
.github/ISSUE_TEMPLATE/*.md	Issue templates.	Standardizes bug reports, feature requests, and rule requests.

6. Installation Commands for Kali and Parrot

The normal user flow is designed to be short. Users clone the repository, enter it, run the installer, reload their shell, and verify the application with doctor.

Command	What it does	Why it is needed
git clone https://github.com/waltr0/smartTerminal.git	Downloads the GitHub repository to the local machine.	Gets the source, installer, docs, package metadata, and packaged data.
cd smartTerminal	Moves the terminal into the project folder.	The installer and package files are located there.

Command	What it does	Why it is needed
bash install.sh	Runs the Linux installer.	Creates a private app venv, installs CyberShell, writes cybershell/cshell launchers, installs shell integration files, and runs doctor.
bash install.sh --shell bash	Installs and configures only Bash integration.	Useful when the user does not use Zsh.
bash install.sh --shell zsh	Installs and configures only Zsh integration.	Useful for Parrot or Kali users who prefer Zsh.
bash install.sh --shell all	Installs both Bash and Zsh integration.	Best for systems where both shells are used.
source ~/.bashrc	Reloads Bash configuration.	Makes shell integration available in the current terminal.
source ~/.zshrc	Reloads Zsh configuration.	Makes Zsh widgets and key bindings available.
cybershell doctor	Checks runtime readiness.	Confirms packaged data and command records are available after install.
cshell doctor	Runs the same check through the short alias.	Confirms both launchers work.

7. Everyday Usage Commands

Command	Explanation
cybershell suggest --partial "show ssh failed logins"	Turns an intent into a safe defensive command. The response includes suggestion, source, why, and risk result.
cybershell risk --mode strict -- "curl http://example.com/install.sh bash"	Scores a command without executing it. In strict mode, remote download piped into shell is blocked.
cybershell explain -- "sudo rm -rf /"	Explains risk findings and safer alternatives for a command.
cybershell kb-search "ssh logs" --json	Searches packaged command records and prints structured JSON results.
cybershell policies	Lists policy modes and thresholds.
cybershell rules	Lists packaged guardrail rules.
cybershell playbook list	Lists built-in defensive workflows.
cybershell playbook show ssh-bruteforce-triage	Shows step-by-step commands for SSH brute-force investigation.
cybershell history-audit --history-file ~/.bash_history	Scans shell history for risky command patterns.
cybershell audit-report	Summarizes accepted suggestions from the local audit file.
cybershell bench-eval --fail-on-miss	Runs the benchmark and exits non-zero if accuracy expectations fail.
cybershell backends	Reports optional FAISS, embedding, numpy, and local GGUF backend availability.
cybershell interactive	Starts a small interactive suggestion loop.
bash uninstall.sh	Removes launchers, shell integration blocks, and application files.

8. Developer and Verification Commands

These commands are useful for maintainers before pushing a release or accepting a contribution.

```
python -m unittest discover -s tests -v
PYTHONPATH=src python -m cybershell bench-eval --fail-on-miss
bash -n install.sh
bash -n uninstall.sh
bash -n scripts/cybershell.bash
python -c "import toml; toml.load(open('pyproject.toml', 'rb')); print('pyproject OK')"
python -m pip install .
cybershell doctor
```

The corrected GitHub repository was validated with 28 unit tests, CyberShell-Bench, shell script syntax checks, pyproject parsing, and an installed-package doctor/benchmark check.

9. Distribution Scope

Item	GitHub status	Reason
Application source	Included	Needed for installation and open-source review.
Installer and uninstaller	Included	Needed by Kali, Parrot, and Linux users.
Tests and benchmark	Included	Needed for validation and research credibility.
PDF guide	Included	Needed for project explanation and install instructions.
Marketing videos	Excluded	External digital marketing asset; not needed for installing the application.

10. Safety and Ethics

CyberShell is built for defensive and authorized use. It does not execute commands for the user. It suggests, scores, explains, and blocks display of unsafe suggestions. High-risk commands should only be used in environments where the user has explicit permission. The guardrail model is not a replacement for professional judgment, but it reduces common terminal safety mistakes.

The strongest safety features are local risk rules, policy thresholds, blocked suggestion suppression, safer alternatives, audit summaries, and benchmark regression checks. These make the project publishable as a cybersecurity education and defensive operations tool rather than an offensive automation framework.

11. Suggested GitHub Release Flow

```
git clone https://github.com/waltr0/smartTerminal.git
cd smartTerminal
bash install.sh
cybershell doctor
cybershell suggest --partial "show ssh failed logins"
cybershell risk --mode strict -- "curl http://example.com/install.sh | bash"
```

For GitHub releases, keep source code, docs, tests, CI, installer, shell integrations, and the PDF guide in the repository. Keep videos and other large external marketing assets outside the source repository, or publish them separately through release assets, a website, or a social platform.

12. Quick Troubleshooting

Problem	Fix
cybershell command not found	Ensure ~/.local/bin is on PATH, restart the terminal, or run source ~/.bashrc / source ~/.zshrc.
python3-venv missing	Install the OS venv package, then rerun bash install.sh. Kali usually supports python3 -m venv once the package exists.
Shell shortcut not active	Reload the shell rc file or open a new terminal.
Benchmark fails	Do not publish until rule, policy, or dataset changes are corrected.
Optional AI backends unavailable	Core app still works. Install optional research extras only when local LLM or FAISS features are required.

Appendix: Feature Checklist

Feature	Status
Offline command knowledge base	Included
Risk scoring and block/warn/allow decisions	Included
MITRE-style tactic and technique labels	Included
Policy modes	Included
Bash and Zsh shell integration	Included
Playbooks	Included
History audit and audit report	Included
CyberShell-Bench	Included
GitHub CI and issue templates	Included
Linux/Kali installer and uninstaller	Included
External marketing video	Excluded from GitHub repository